```
const { onDocumentCreated } =
require("firebase-
functions/v2/firestore");
const { onSchedule } =
require("firebase-
functions/v2/scheduler");
const { initializeApp } =
require("firebase-admin/app");
const { getFirestore, FieldValue,
Timestamp } = require("firebase-
admin/firestore");
const { getMessaging } =
require("firebase-
admin/messaging");

initializeApp();
const db = getFirestore();

const APP_URL =
process.env.APP_URL ||
"https://saastunbienesraices-
oss.github.io/saastun_crm/";
const TZ = "America/Merida";
```

```
const INACTIVO_DIAS =
Number(process.env.INACTIVO_DIAS
|| 3);
const REASIGNAR_DIAS =
Number(process.env.REASIGNAR_DIAS
|| 0);
const WA_TIPOS =
["cierre_pendiente",
"negociacion_alert",
"inactivo_resumen",
"sin_actividad"];

// ── WhatsApp (CallMeBot) ──
async function
enviarWhatsApp(texto) {
  const phone =
process.env.WA_PHONE, key =
process.env.WA_APIKEY;
  if (!phone || !key) return;
  try {
    const url =
`https://api.callmebot.com/whatsa
pp.php?
phone=${encodeURIComponent(phone)
}&text=${encodeURIComponent(texto
)}&apikey=${encodeURIComponent(ke
```

```
y)}`;
    const r = await fetch(url);
    if (!r.ok)
console.warn("WhatsApp",
r.status, await r.text());
  } catch (e) {
console.warn("WhatsApp",
e.message); }
}

// ── Cada notificación nueva →
push al teléfono (app cerrada) +
WhatsApp al jefe ──
exports.enviarAviso =
onDocumentCreated("notificaciones
/{id}", async (event) => {
  const n = event.data &&
event.data.data();
  if (!n) return;

  const destinos = new Map();
  if (n.paraUid) {
    const s = await
db.collection("usuarios").doc(n.p
araUid).get();
    if (s.exists)
```

```
destinos.set(s.id, s.data());
  }
  if (n.paraAdmin) {
    const q = await
db.collection("usuarios").where("
rol", "==", "admin").get();
    q.forEach((d) =>
destinos.set(d.id, d.data()));
  }
  if (n.por)
destinos.delete(n.por); // no
avisar a quien hizo la acción

  const tokens = [], dueno = {};
  destinos.forEach((u, uid) =>
(u.fcmTokens || []).forEach((t)
=> { tokens.push(t); dueno[t] =
uid; }));

  if (tokens.length) {
    const res = await
getMessaging().sendEachForMultica
st({
      tokens,
      notification: { title:
"Sáastun CRM", body: n.texto ||
```

```
"" },
      webpush: {
        notification: { icon:
APP_URL + "icon-1024.png", tag:
n.tipo || "aviso" },
        fcmOptions: { link:
APP_URL },
      },
      data: { leadId: n.leadId ||
"", tipo: n.tipo || "" },
    });
    // limpiar tokens que ya no
sirven
    const malos = {};
    res.responses.forEach((r, i)
=> {
      const code = r.error &&
r.error.code;
      if (!r.success && (code ===
"messaging/registration-token-
not-registered" || code ===
"messaging/invalid-registration-
token")) {
        const uid =
dueno[tokens[i]];
        (malos[uid] = malos[uid]
```

```
|| []).push(tokens[i]);
      }
    });
    await
Promise.all(Object.entries(malos)
.map(([uid, arr]) =>

db.collection("usuarios").doc(uid
).update({ fcmTokens:
FieldValue.arrayRemove(...arr)
})));
  }

  if (n.paraAdmin &&
WA_TIPOS.includes(n.tipo)) {
    await enviarWhatsApp("Sáastun
CRM: " + (n.texto || ""));
  }
});

// ── utilidades ──
function dias(ts) {
  const d = ts && ts.toDate ?
ts.toDate() : null;
  return d ?
Math.floor((Date.now() -
```

```
d.getTime()) / 864e5) : 999;
}
function avisar(texto, { paraUid
= null, paraAdmin = false, tipo,
leadId = null }) {
  return
db.collection("notificaciones").a
dd({
    texto, paraUid, paraAdmin,
tipo, leadId, leida: false,
    fecha:
FieldValue.serverTimestamp(),
por: "sistema",
  });
}
// Mérida está en UTC-6 todo el
año (sin horario de verano)
function inicioDiaLocal() {
  const f = new
Intl.DateTimeFormat("en-CA", {
timeZone: TZ, year: "numeric",
month: "2-digit", day: "2-digit"
}).format(new Date());
  return new Date(f + "T00:00:00-
06:00");
}
```

```
// ── 9:00 am: clientes sin atención (+ reasignación si está activada) ──
exports.revisionMatutina = onSchedule({ schedule: "every day 09:00", timeZone: TZ }, async () => {
  const [lq, uq] = await Promise.all([db.collection("leads").get(), db.collection("usuarios").get()]);
  const asesores = uq.docs.filter((d) => d.data().rol !== "admin" && d.data().activo !== false);
  const nombres = {};
  uq.forEach((d) => (nombres[d.id] = d.data().nombre));
  const activos = lq.docs.filter((d) => !["ganado", "perdido"].includes(d.data().etapa) && d.data().asesorId);
```

```
  const carga = {};
  asesores.forEach((a) =>
(carga[a.id] = 0));
  activos.forEach((d) => { const
a = d.data().asesorId; if (a in
carga) carga[a]++; });

  const porAsesor = {}, movidos =
[];
  for (const d of activos) {
    const l = d.data();
    const inact =
dias(l.actualizado || l.creado);

    if (REASIGNAR_DIAS > 0 &&
inact >= REASIGNAR_DIAS && inact
< 900 &&
        dias(l.reasignadoEn) >=
REASIGNAR_DIAS && movidos.length
< 10) {
      const cand =
asesores.filter((a) => a.id !==
l.asesorId).sort((x, y) =>
carga[x.id] - carga[y.id])[0];
      if (cand) {
        if (l.asesorId in carga)
```

```
carga[l.asesorId]--;
        carga[cand.id]++;
        await d.ref.update({
asesorId: cand.id, reasignadoEn:
FieldValue.serverTimestamp(),
actualizado:
FieldValue.serverTimestamp() });
        await
d.ref.collection("historial").add
({
          texto: `Reasignado
automáticamente de
${nombres[l.asesorId] || "?"} a
${cand.data().nombre} (${inact}
días sin atención)`,
          ico: "🔄", fecha:
FieldValue.serverTimestamp(),
usuario: "Sistema",
        });
        await avisar(`Te
asignaron a "${l.nombre}"
(llevaba ${inact} días sin
atención)`, { paraUid: cand.id,
tipo: "reasignado", leadId: d.id
});
```

```
movidos.push(`${l.nombre}:
${nombres[l.asesorId] || "?"} →
${cand.data().nombre}`);
        continue;
      }
    }
    if (inact >= INACTIVO_DIAS)
(porAsesor[l.asesorId] =
porAsesor[l.asesorId] ||
[]).push(l.nombre);
  }

  for (const [uid, lista] of
Object.entries(porAsesor)) {
    await avisar(`Tienes
${lista.length} cliente(s) sin
atención: ${lista.slice(0,
5).join(", ")}${lista.length > 5
? "…" : ""}`,
      { paraUid: uid, tipo:
"inactivo_resumen" });
  }
  const resumen =
Object.entries(porAsesor).map(([u
id, l]) => `${nombres[uid] ||
"?"}: ${l.length}`);
```

```
  if (resumen.length ||
movidos.length) {
    await avisar(`Clientes sin
atención → ${resumen.join(" · ")
|| "ninguno"}${movidos.length ? "
| Reasignados: " +
movidos.join("; ") : ""}`,
      { paraAdmin: true, tipo:
"inactivo_resumen" });
  }
});

// ── 6:30 pm: asesores sin
actividad en el día ──
exports.revisionVespertina =
onSchedule({ schedule: "every day
18:30", timeZone: TZ }, async ()
=> {
  const [aq, uq] = await
Promise.all([

db.collection("actividad").where(
"fecha", ">=",
Timestamp.fromDate(inicioDiaLocal
())).get(),
```

```
db.collection("usuarios").get(),
  ]);
  const con = new
Set(aq.docs.map((d) =>
d.data().uid));
  const sin = uq.docs.filter((d)
=> d.data().rol !== "admin" &&
d.data().activo !== false &&
!con.has(d.id)).map((d) =>
d.data().nombre);
  if (sin.length) await
avisar(`Hoy sin actividad:
${sin.join(", ")}`, { paraAdmin:
true, tipo: "sin_actividad" });
});
```